

PilotFish Http Response Status Code Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

For synchronous HTTP routes, this step **chooses the HTTP status code** (200 OK, 404 Not Found, etc.) and stores just the numeric code on the transaction. The HTTP listener reads that value when it sends the response back to the client.

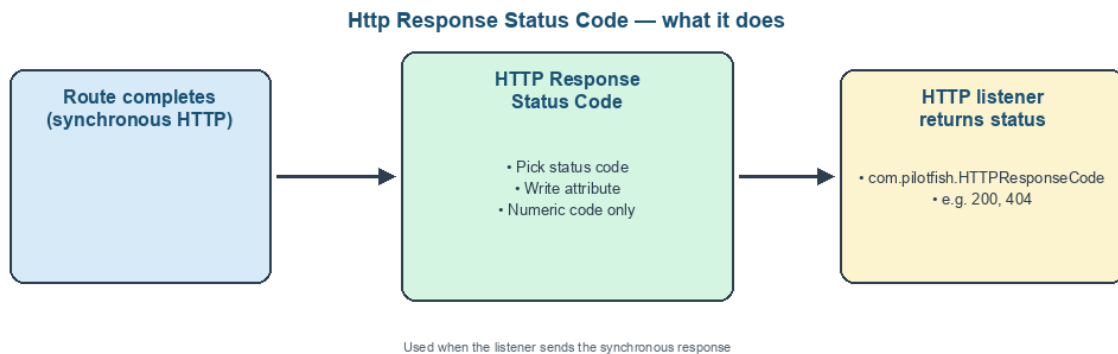


Figure 1: At a glance (plain English)

- **Selects** a status from the standard HTTP status list.
- **Writes** the numeric code to `com.pilotfish.HTTPResponseCode`.
- **Does not** set headers or body — pair with other HTTP processors.
- **Used by** HTTP listeners on synchronous response paths.

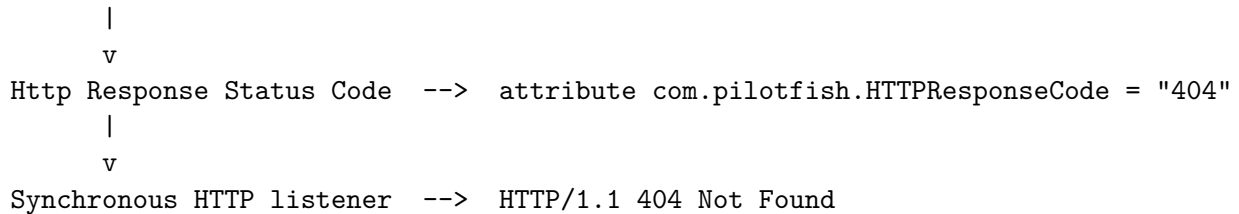
Purpose

This document is a **deep dive** into the **Http Response Status Code** processor as shipped in `eip.war.hs.26R1.11`.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-http-1.0.3.jar</code>
Decompiler	CFR 0.152
Implementation class	<code>com.pilotfish.eip.modules.http.HttpResponseStatusProcessor</code>
Processor type string	Http Response Status Code
Description	Set the status code for the HTTP response in a transaction attribute that will be used by HTTP listeners when they receive the synchronous response.
Categories	HTTP, WEB

1. Conceptual model

Route logic (maybe branch on error)



2. High-level behavior (processData)

1. Read configured **Status Code** string: `manager.getStringValue("StatusCode", data)`.
2. Split on first space: `statusCodeString.split(" ")[0]` -> numeric code only.
 - UI stores values like "200 OK", "404 Not Found".
3. `data.getAttributes().setAttribute("com.pilotfish.HTTPResponseCode", code)`.
4. Return data.

No stream or header changes.

3. Configuration reference

3.1 Status Code

Property	Value
UI label	Status Code
Tag	<code>StatusCode</code>
Type	Multiple choice
Choices	All jakarta.ws.rs.core.Response.Status enum values formatted as <code>{code}</code> <code>{reasonPhrase}</code>
Default	200 OK
Tooltip	The status code to return in the HTTP response.

Nuances:

- Static initializer iterates `Response.Status.values()` at class load — the list matches JAX-RS status constants shipped with the WAR.
- Only the **first token** (numeric string) is stored; reason phrase is discarded at runtime.
- Dynamic/runtime selection requires a different pattern (e.g. Attribute Population setting `HTTPResponseCode` directly).

3.2 Inherited AbstractProcessor options

Tag | `ExecuteProcessor` | Default `true` |

4. Transaction attributes

Attribute key	Type	Example
<code>com.pilotfish.HTTPResponseCode</code>	<code>String</code> (numeric)	"200", "500"

5. Worked examples

5.1 Success path

Status Code = 200 OK -> attribute "200".

5.2 Not found branch

Place this processor on an error branch with Status Code = 404 Not Found -> attribute "404".

6. Known quirks and caveats

1. **Fixed dropdown only** — no built-in expression field; value is static per processor configuration unless multiple conditional processor instances are used.
 2. **Split on space** — malformed stored value without a space would store the entire string as the code.
 3. **Listener-dependent** — attribute has effect only when the originating HTTP listener honors synchronous response attributes.
 4. **Reason phrase not stored** — listeners typically map numeric code back to a standard phrase.
-

7. Operational recommendations

Goal	Guidance
REST APIs	Place one instance per outcome branch (2xx/4xx/5xx)
With headers	Combine with Http Response Headers processor
Dynamic codes	Set <code>com.pilotfish.HTTPResponseCode</code> via Attribute Population instead
Testing	Verify listener synchronous mode in eiConsole test harness

8. Source index (this WAR)

Topic	Path
Http Response Status Code	<code>com/pilotfish/eip/modules/http/HttpResponseCodeProcessor</code>
Module JAR	<code>xcs-modules-http-1.0.3.jar</code>

9. Pipeline provenance

PilotFish WARs/eip.war.hs.26R1.11 -> xcs-modules-http-1.0.3.jar -> CFR -> Documents/Processors

Deep-dive documentation – Http Response Status Code Processor – 26R1.11 – August 2026.